

← → adaptiveresumescreener.com 🔍

ADAPTIVE RESUME SCREENER

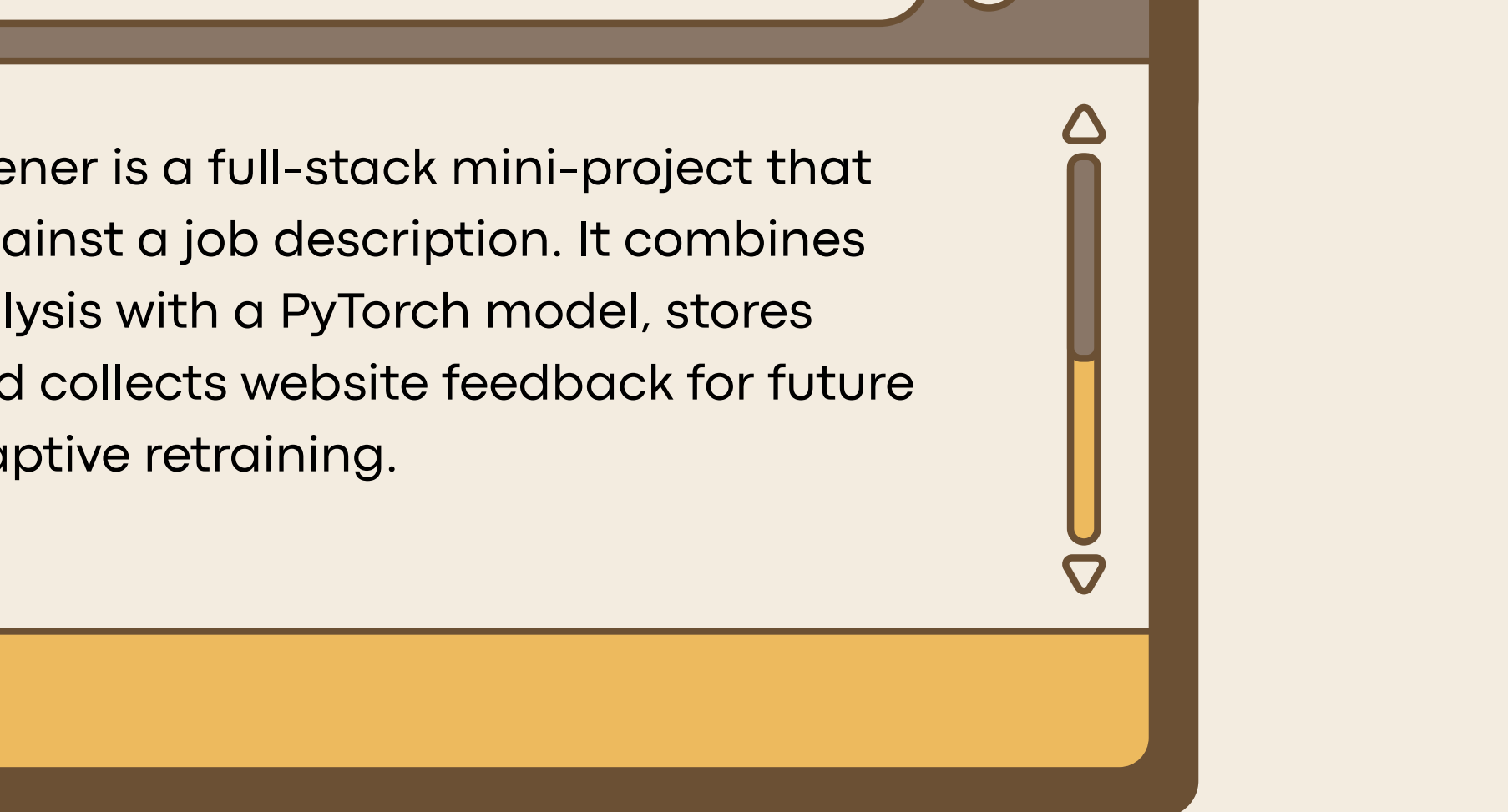
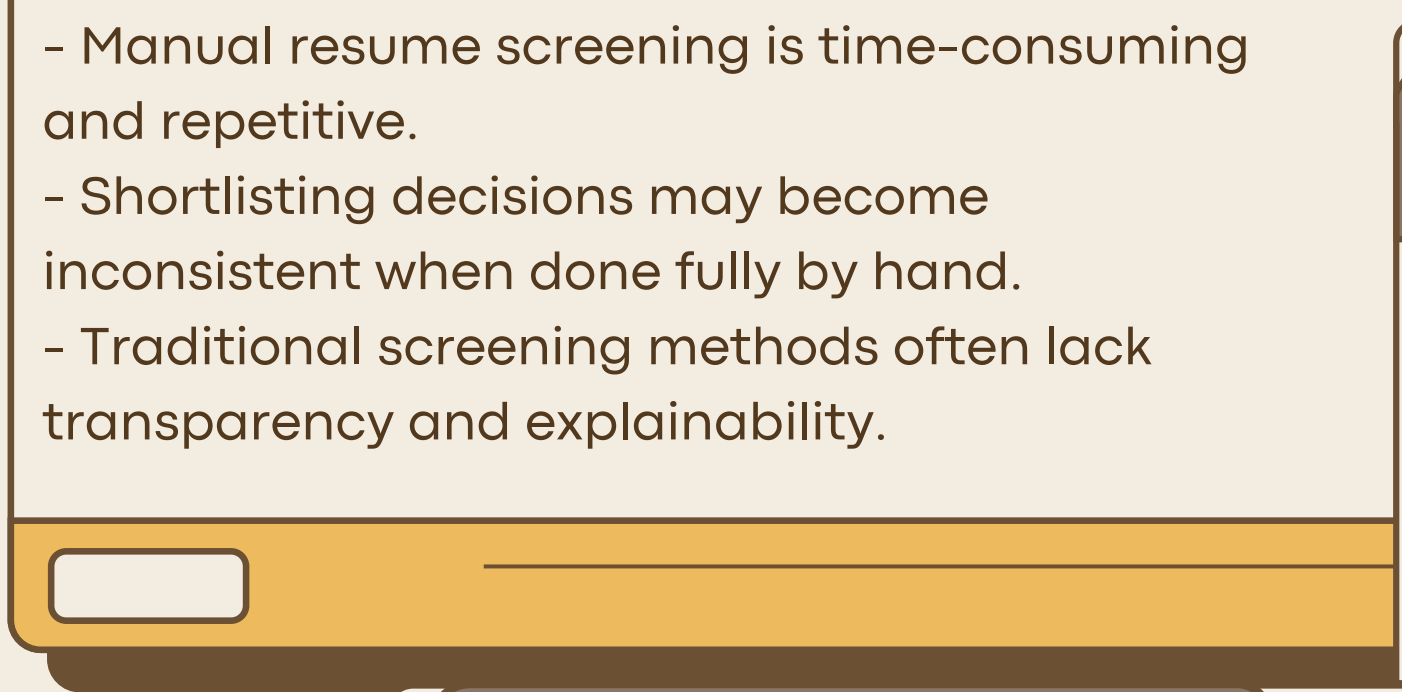
Team members:

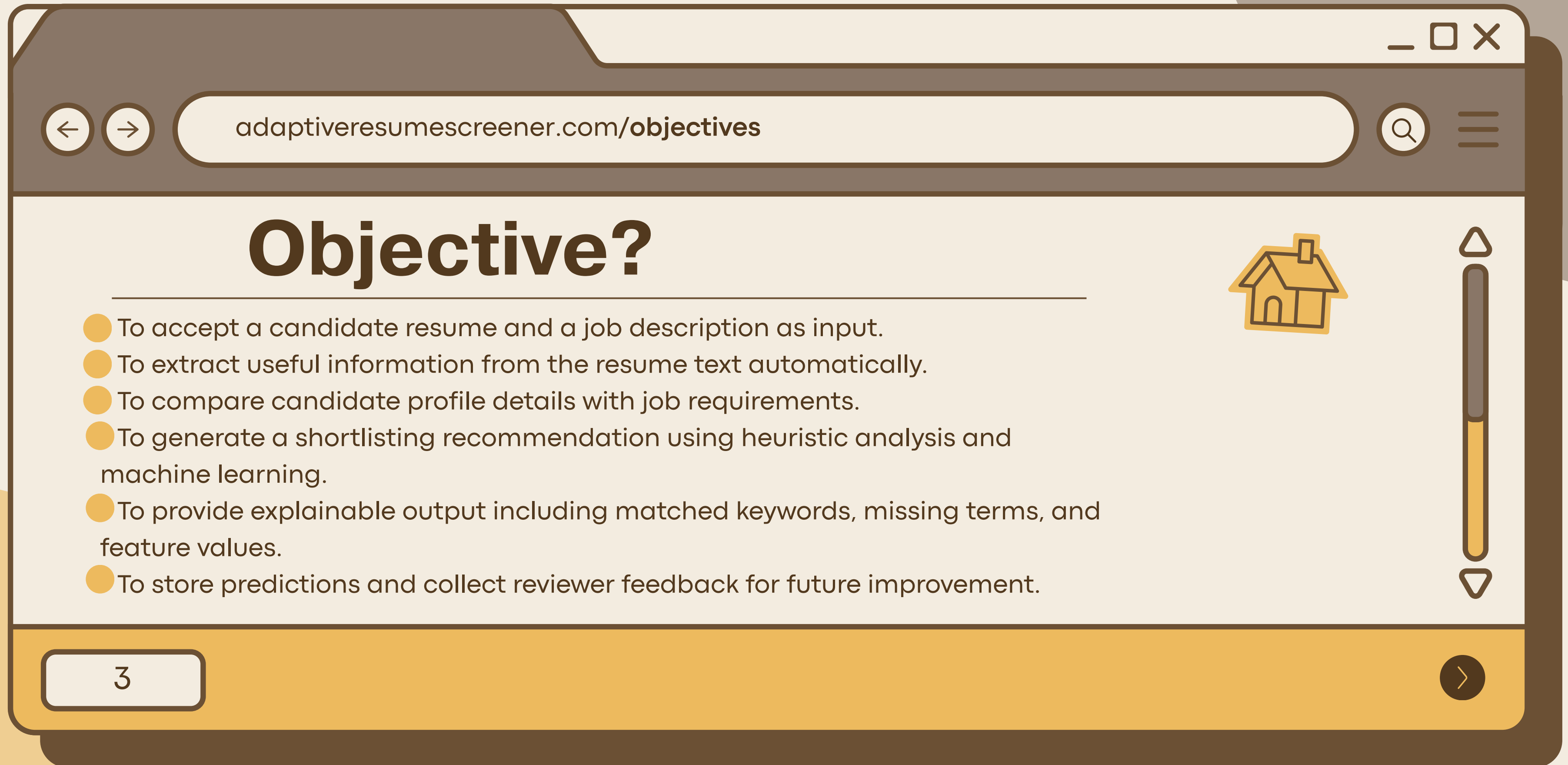
Mrinalendu Maji	2305139
Ritik Mohanty	2305320
Shubh Agnihotri	2305336
Aditya Chandel	2305678
Harshit Belwal	2305702

Guided by:

Mr. Vijay Kumar Meena

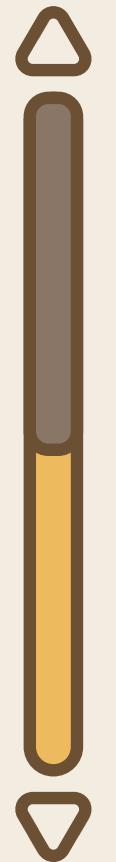




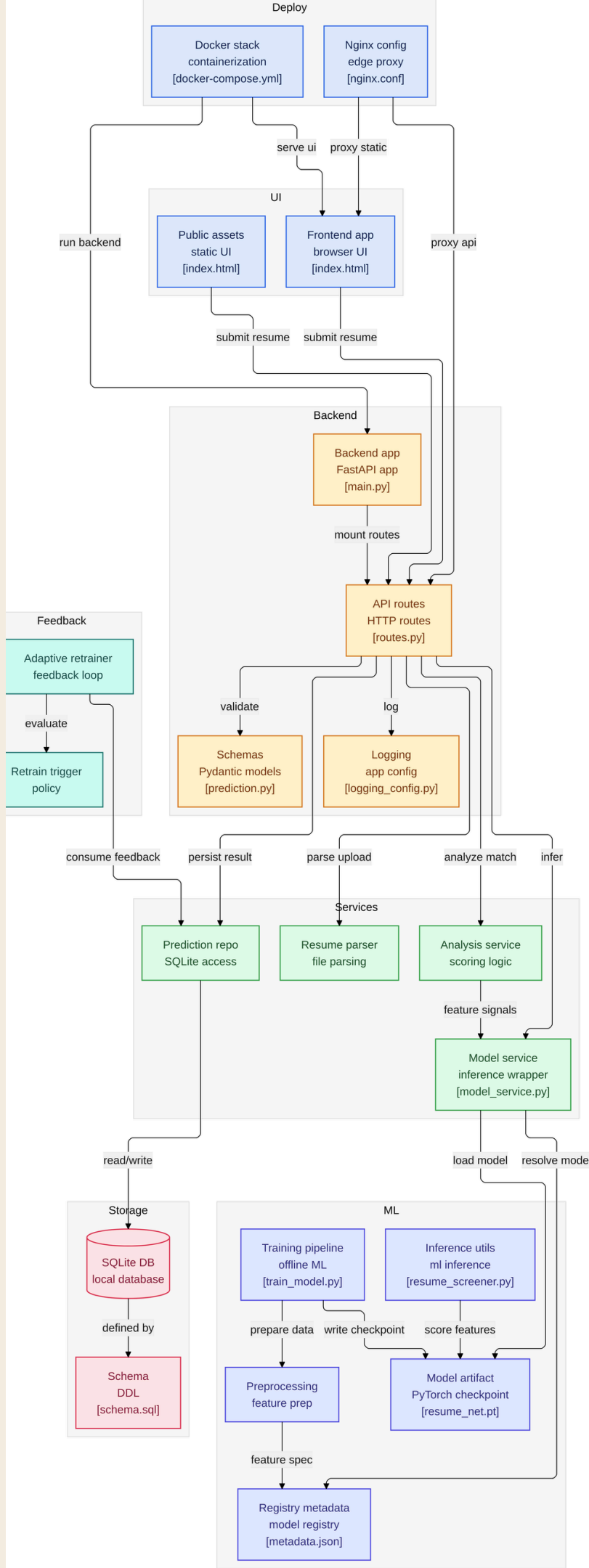
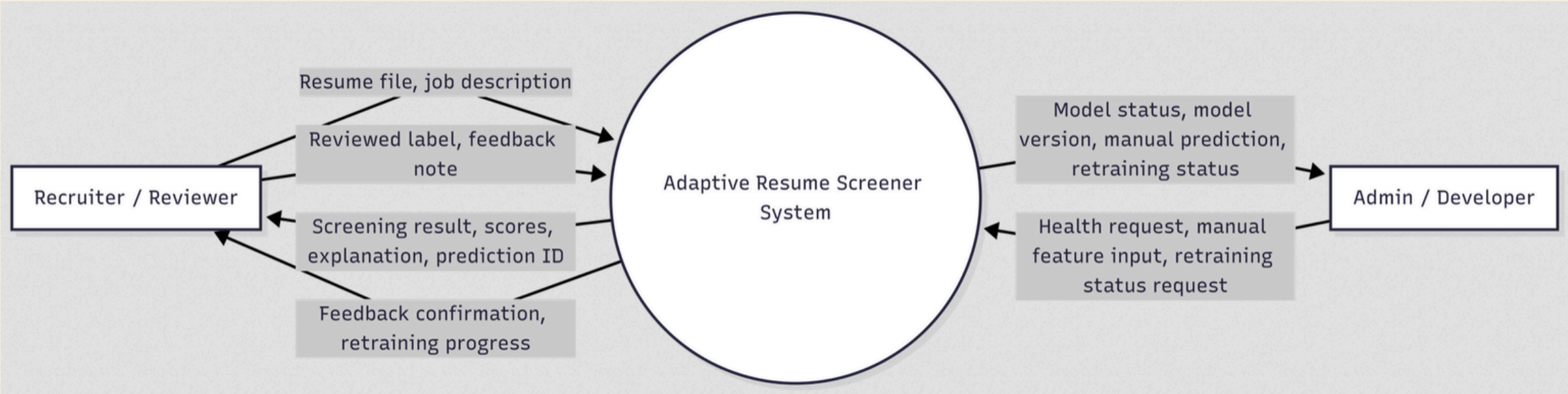


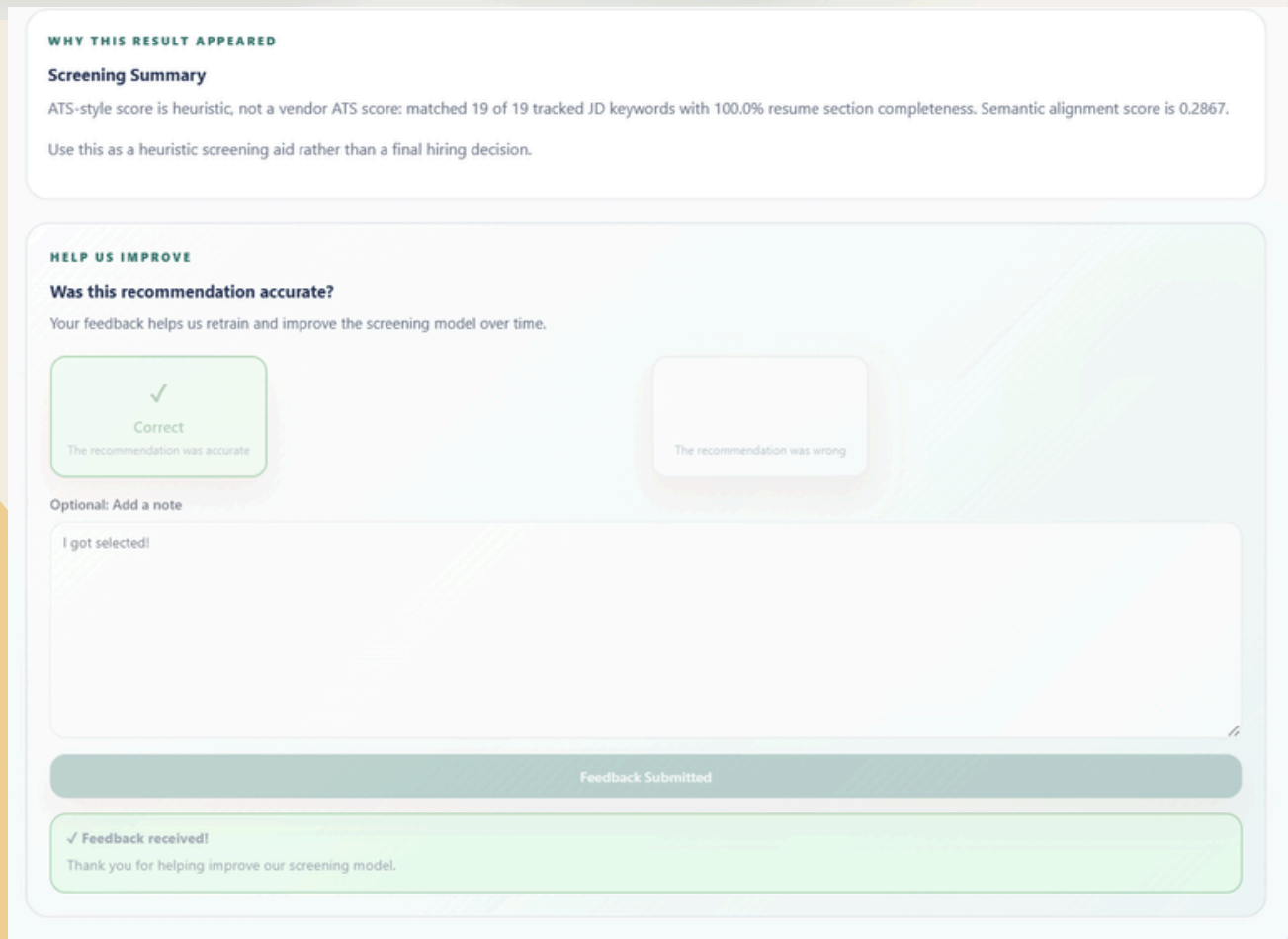
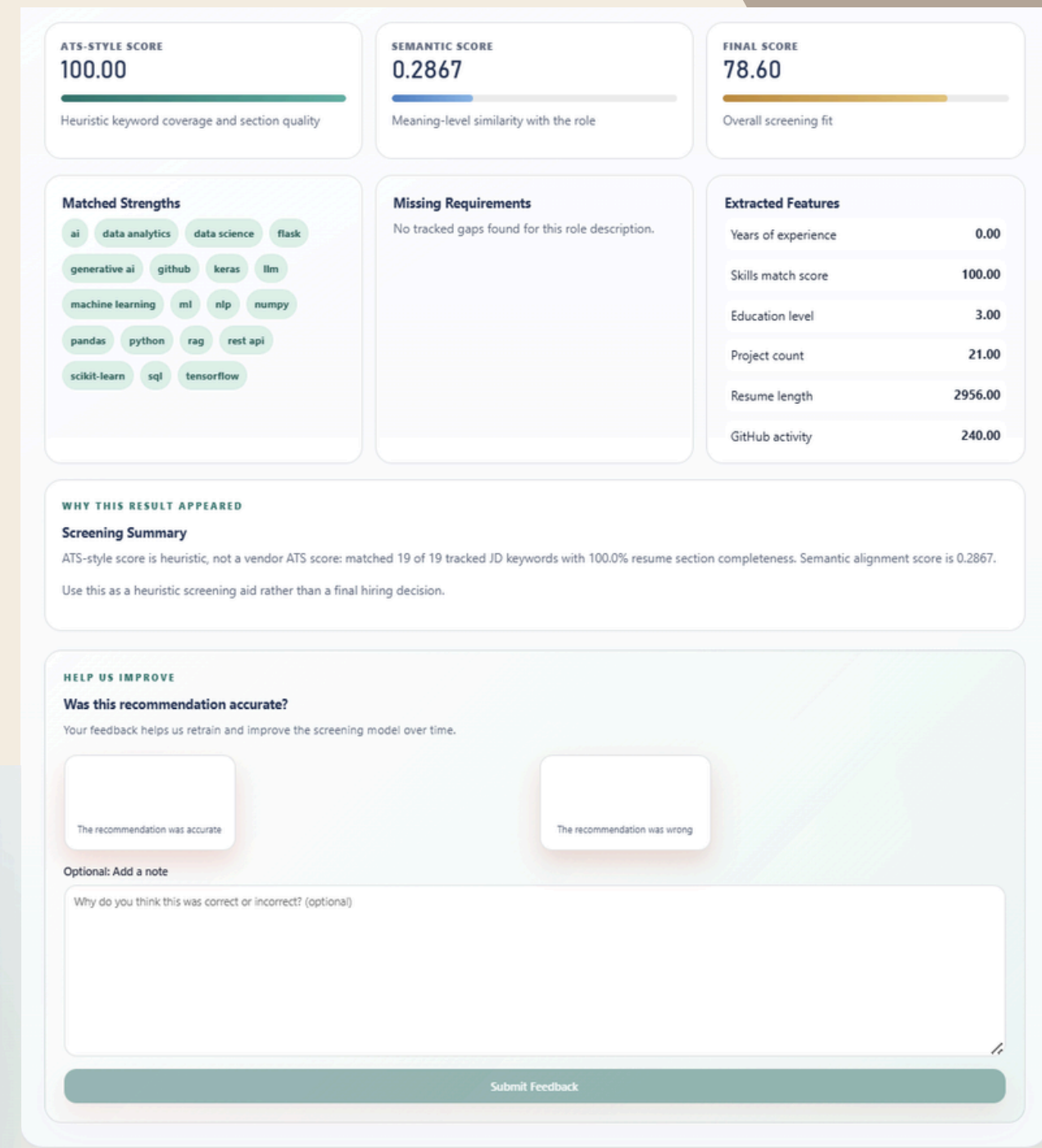
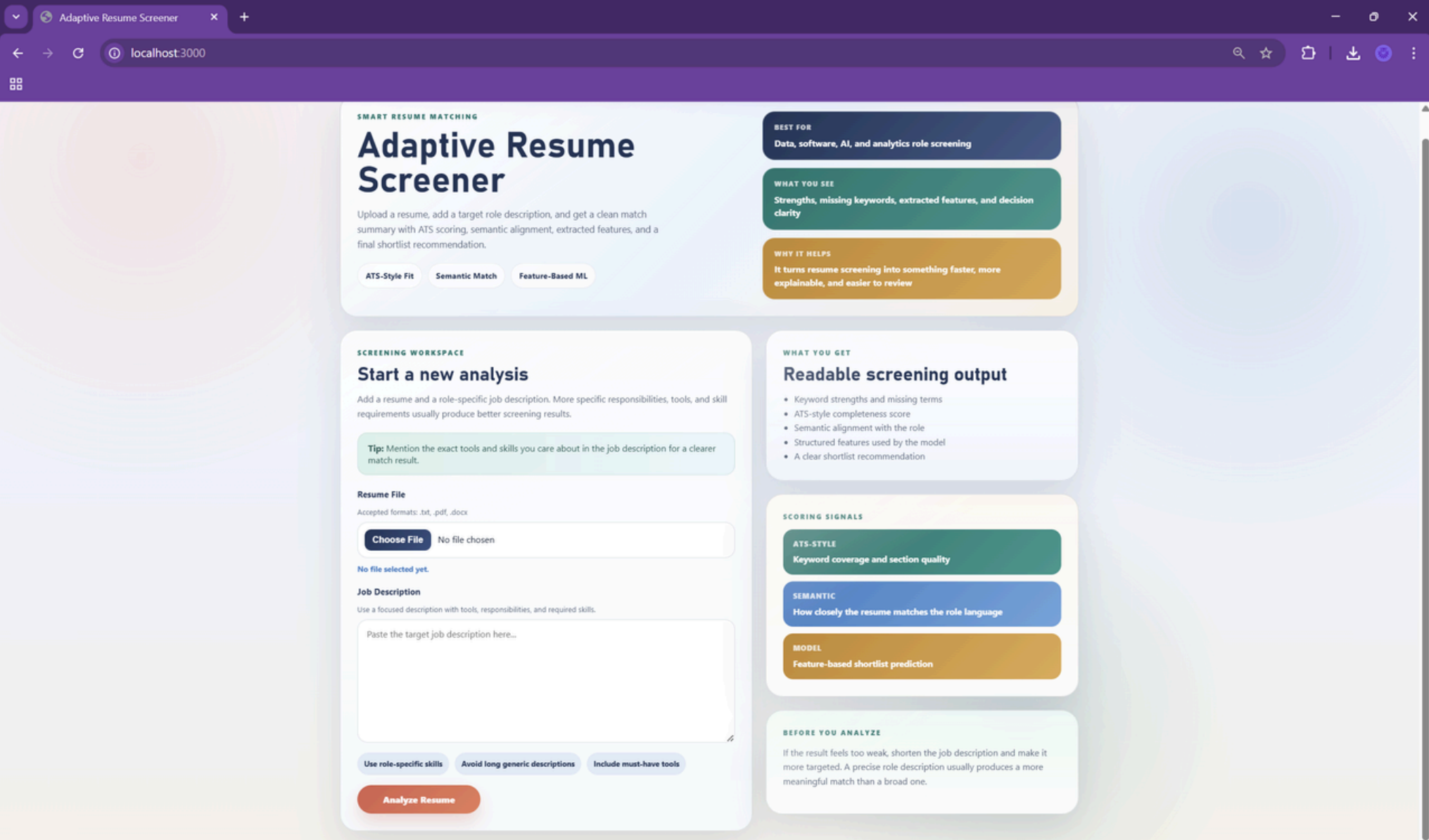
Objective?

- To accept a candidate resume and a job description as input.
- To extract useful information from the resume text automatically.
- To compare candidate profile details with job requirements.
- To generate a shortlisting recommendation using heuristic analysis and machine learning.
- To provide explainable output including matched keywords, missing terms, and feature values.
- To store predictions and collect reviewer feedback for future improvement.



- Frontend collects the resume file and job description from the user.
- Backend API processes the request and coordinates all services.
- Resume analysis generates explainable features and matching signals.
- ResumeNet ML model predicts shortlist suitability.
- SQLite stores predictions, feedback, and retraining readiness data.

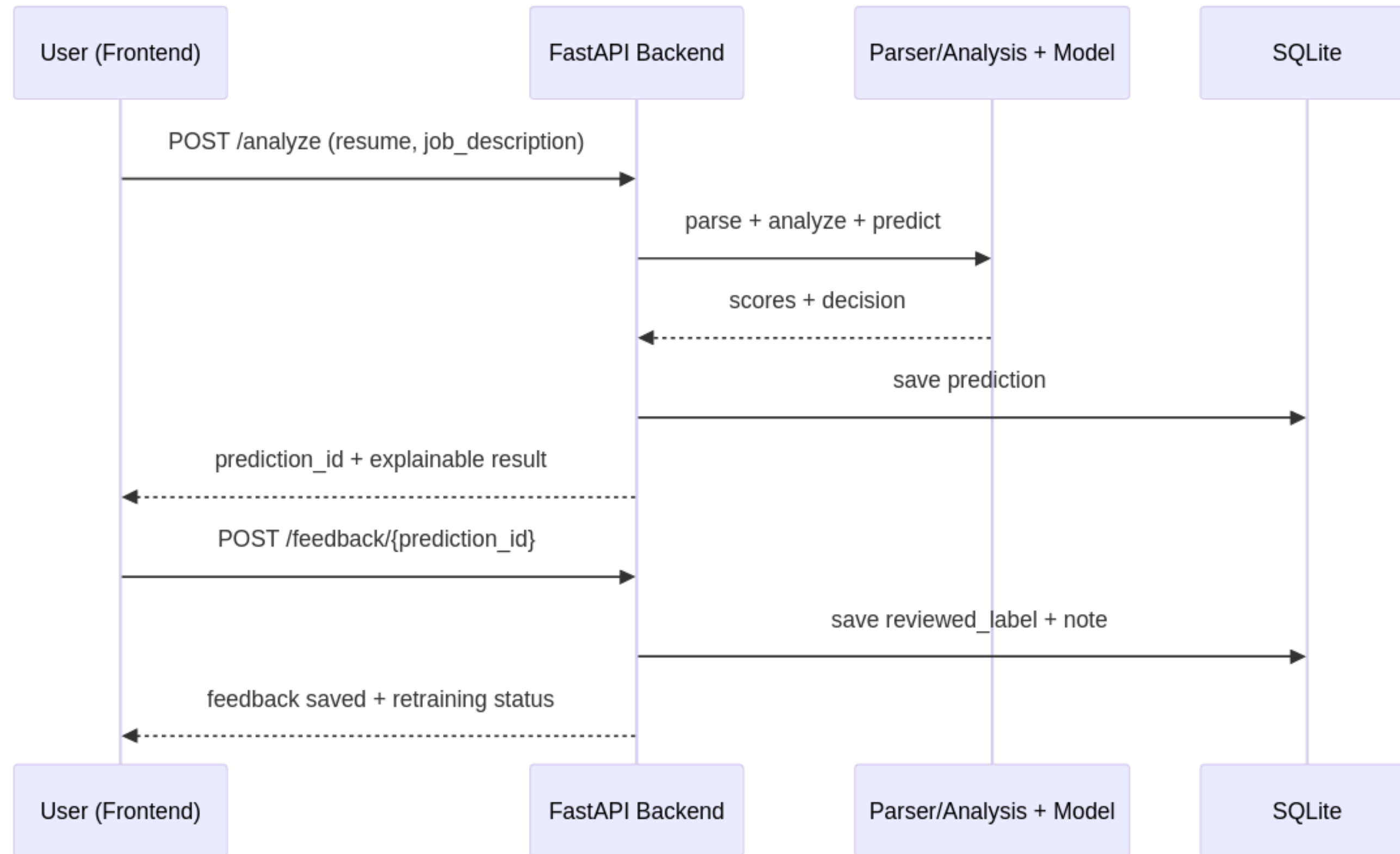


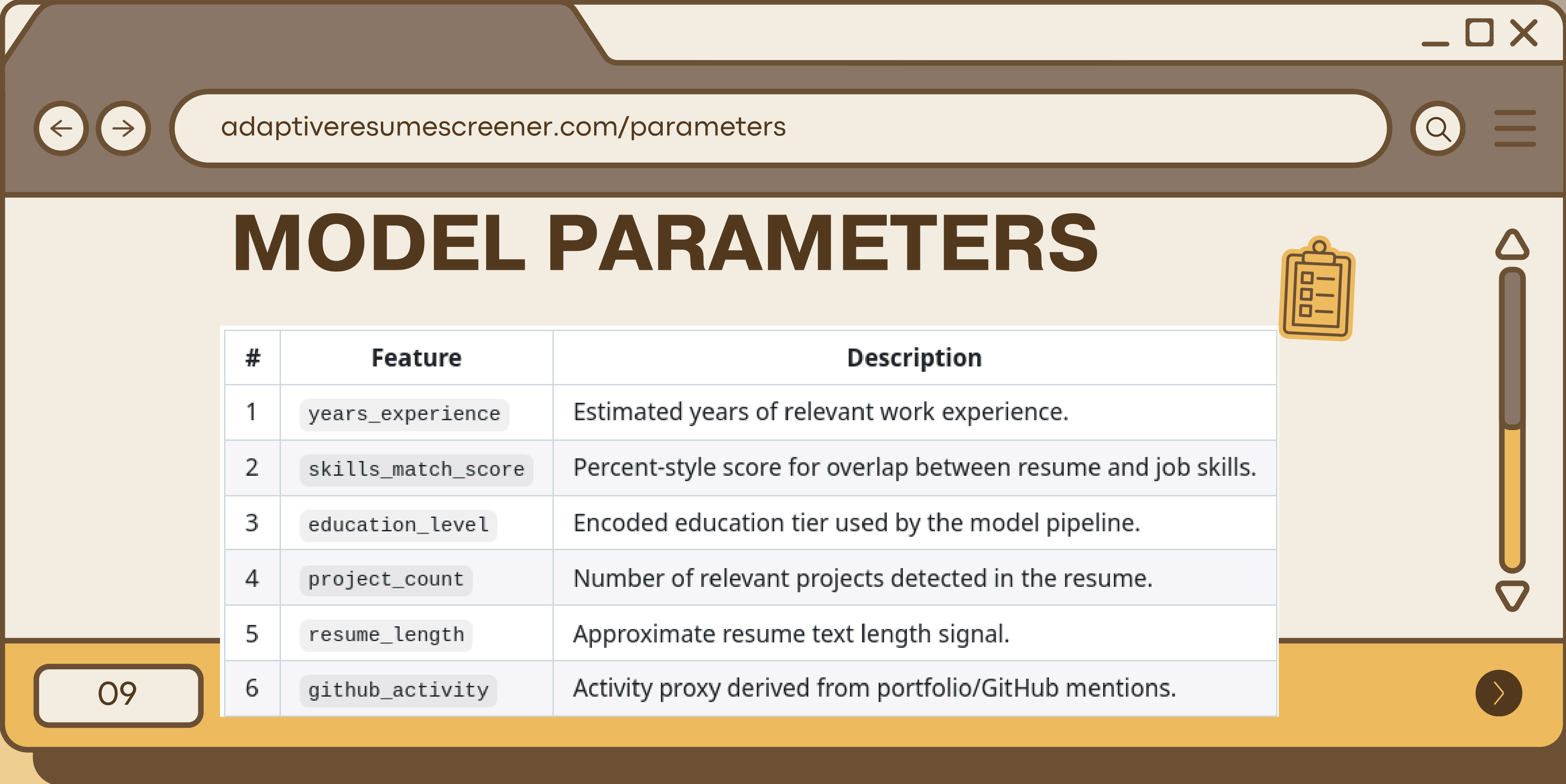


No	Feature Name	Description	Range
1	Years of Experience	Maximum years of experience mentioned in the resume, extracted using regex and capped before model use.	0 - 60
2	Skills Match Score	Percentage of tracked job-description keywords found in the resume.	0 - 100
3	Education Level	Ordinal encoding inferred from degree keywords such as diploma, bachelor,	0 - 4
4	Project Count	Estimated number of projects inferred from explicit project mentions in the	0 - 100
5	Resume Length	Character length of the cleaned resume text, used as a simple completeness signal.	0 - 10000
6	GitHub Activity	Heuristic score based on GitHub mentions, repository links, and open-source activity	0 - 10000

Method	Endpoint	Description	Response
GET	/health	Returns backend health, model load status, threshold, device, and model	200 OK with health JSON
POST	/predict	Accepts a manual 6-feature payload and runs direct model inference.	200 OK with probability, decision, threshold
POST	/analyze	Accepts resume file and job description, runs parsing, analysis, model inference,	200 OK with prediction_id, scores, keywords, explanation,
POST	/feedback/{prediction_id}	Stores reviewed_label (0/1) and optional feedback_note for an existing prediction	200 OK with feedback status or 404 if record is not found
GET	/feedback/retraining-status	Returns labeled feedback count and retraining readiness status	200 OK with readiness JSON
GET	/docs	Auto-generated Swagger UI documentation for the API .	Interactive HTML documentation

API-ENDPOINTS-FLOW-DIAGRAM

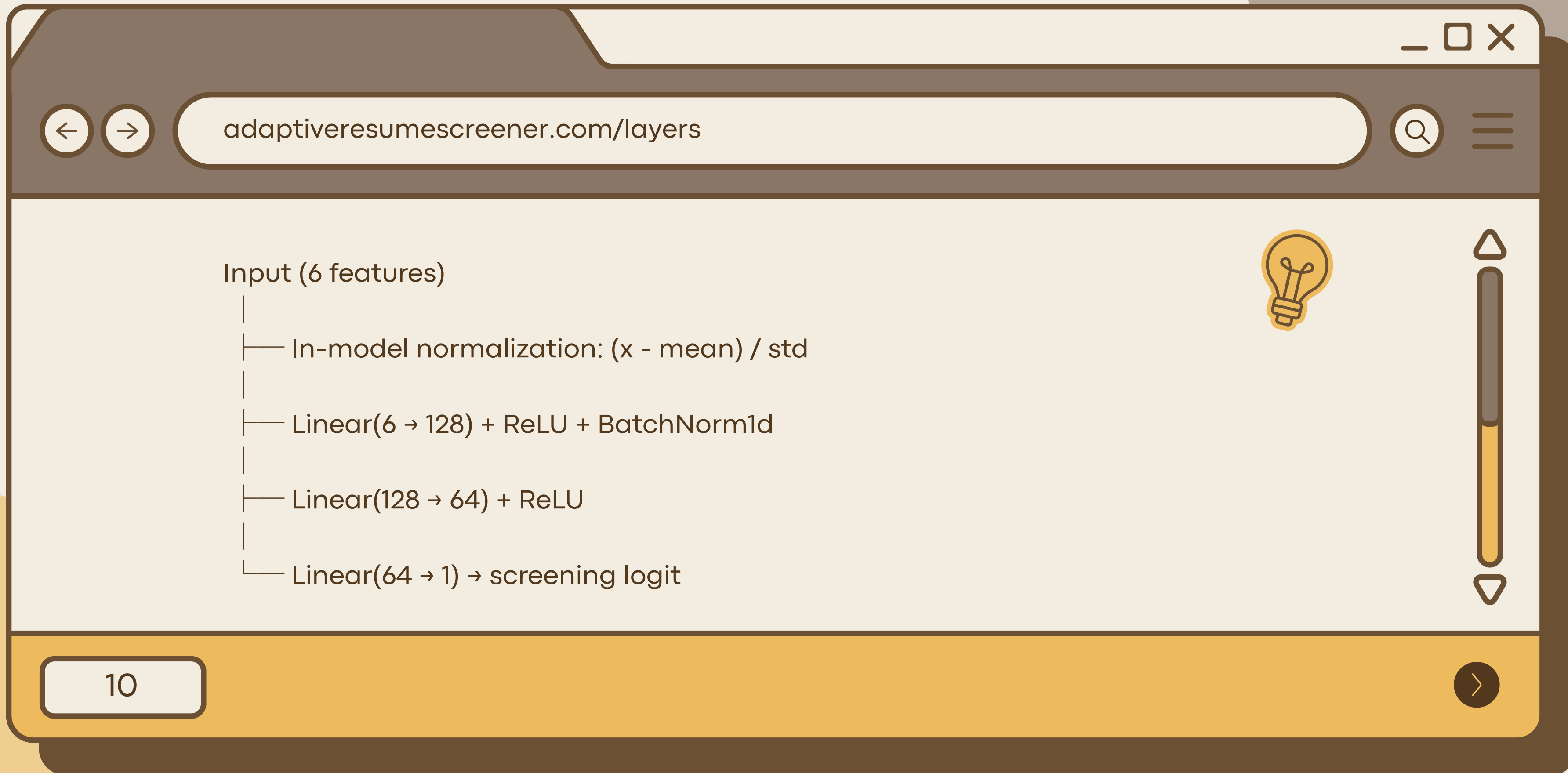




adaptiveresumescreener.com/parameters

MODEL PARAMETERS

#	Feature	Description
1	years_experience	Estimated years of relevant work experience.
2	skills_match_score	Percent-style score for overlap between resume and job skills.
3	education_level	Encoded education tier used by the model pipeline.
4	project_count	Number of relevant projects detected in the resume.
5	resume_length	Approximate resume text length signal.
6	github_activity	Activity proxy derived from portfolio/GitHub mentions.



Input (6 features)

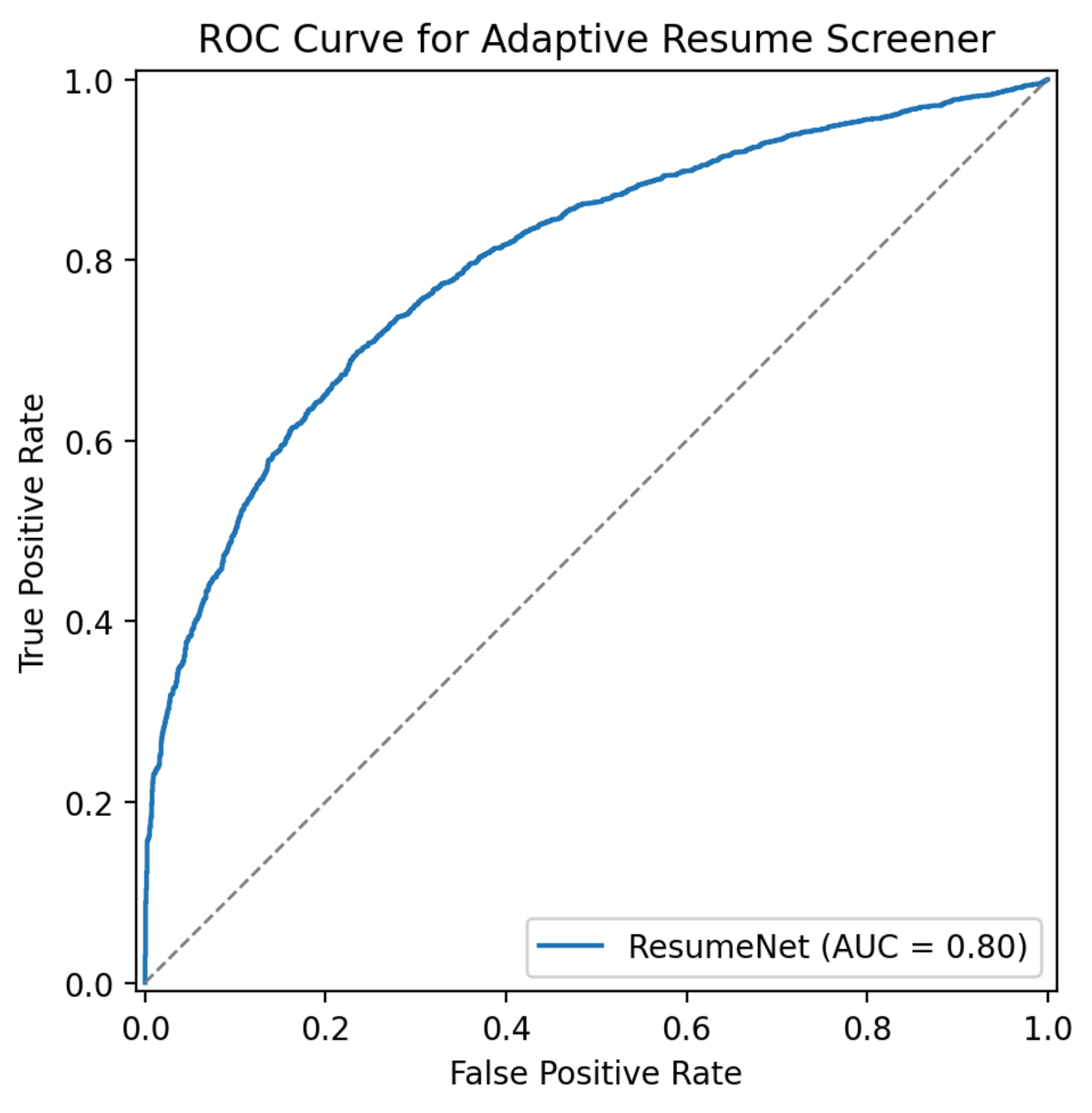
- In-model normalization: $(x - \text{mean}) / \text{std}$
- Linear(6 → 128) + ReLU + BatchNorm1d
- Linear(128 → 64) + ReLU
- Linear(64 → 1) → screening logit



10



PERFORMANCE METRICS



Metric	Value
Threshold	0.30
Accuracy	0.6988
Precision	0.6988
Recall	1.0000
F1	0.8227
AUC-ROC	0.7982

ER DIAGRAM

The app stores predictions and feedback in a single SQLite table: predictions.

What is stored in SQLite

- core model inputs and outputs (years_experience, skills_match_score, education_level, probability, decision, threshold, model_version)
- analysis artifacts for explainability (ats_score, semantic_score, final_score, matched/missing keywords, explanation)
- provenance fields (source, created_at, resume_filename, job_description, resume_excerpt)
- reviewer feedback fields (reviewed_label, reviewed_at, feedback_note)

How feedback is linked to prediction_id

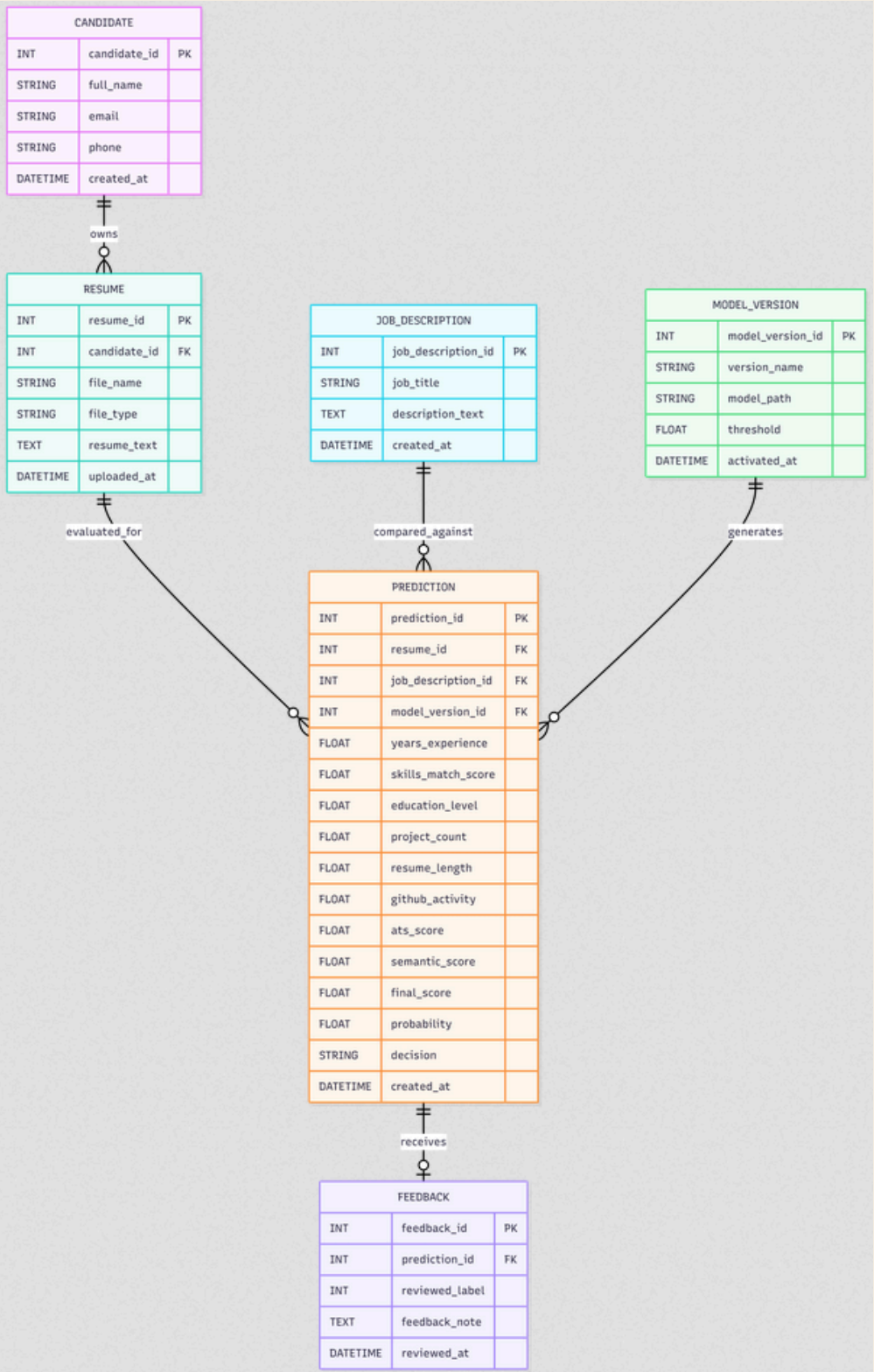
- each prediction insert returns an auto-increment id
- /analyze response returns this value as prediction_id
- /feedback/{prediction_id} updates that same row (WHERE id = ?) with reviewed_label, reviewed_at, and optional feedback_note

How retraining readiness is tracked

- readiness is computed from COUNT(*) WHERE reviewed_label IS NOT NULL
- the retraining trigger compares labeled count vs a configured minimum (default: 100)

/feedback/retraining-statusand feedback responses report:

- ready
- labeled_feedback_count
- minimum_required
- status message





adaptiveresumescreeener.com/future-enhancements



- > Adaptive retraining using more reviewed feedback
- > Improved resume parsing and skill extraction
- > Better model selectivity and threshold tuning
- > Larger dataset and role-specific screening models

